

Project 2 Report: Dynamic Programming Typo Correction

Akshara, Lydia, and Mandar

Spring 2026

Implementation Note: This solution uses a *memoized* top-down dynamic programming approach. It supports the four standard typo operations (match, insert, delete, substitute) as well as two transpose variants: a simple adjacent transposition and a “double-consecutive” transposition (two overlapping adjacent transpositions encoded as a single DP transition). Transpositions are restricted to immediately adjacent character pairs in the DP (i.e. the algorithm does *not* implement unrestricted Damerau–Levenshtein; it uses the Optimal String Alignment variant extended with the double-transpose shortcut).

1 Breaking Down the Problem into Subproblems

We define $\text{solve}(i, j)$ as the minimum cost to transform $\text{target}[i \dots n-1]$ into $\text{typo}[j \dots m-1]$ —that is, the cost of aligning the *suffixes* starting at positions i and j respectively.

The full answer is $\text{solve}(0, 0)$. Each subproblem $\text{solve}(i, j)$ is broken down by choosing which typo operation to apply *first* (leftmost):

- **Match:** if $\text{target}[i] = \text{typo}[j]$, consume both characters for free and recurse on $(i+1, j+1)$.
- **Insert:** insert $\text{typo}[j]$ into the output, pay the insertion cost, and recurse on $(i, j+1)$.
- **Delete:** delete $\text{target}[i]$ from the target, pay the deletion cost, and recurse on $(i+1, j)$.
- **Substitute:** replace $\text{target}[i]$ with $\text{typo}[j]$ (when they differ), pay the substitution cost, and recurse on $(i+1, j+1)$.
- **Transpose (1-swap):** if $\text{target}[i] = \text{typo}[j+1]$ and $\text{target}[i+1] = \text{typo}[j]$, swap the pair, pay the transpose cost, and recurse on $(i+2, j+2)$.
- **Transpose (2-swap):** if $\text{target}[i+1] = \text{typo}[j]$, $\text{target}[i+2] = \text{typo}[j+1]$, and $\text{target}[i] = \text{typo}[j+2]$, perform two consecutive adjacent transpositions in one step, pay both transpose costs, and recurse on $(i+3, j+3)$.

The solution to $\text{solve}(0, 0)$ is assembled by greedily picking the minimum-cost option at each state and combining it with the optimal solution to the resulting subproblem.

2 Parameters of the Recursive Function

The recursive function has signature:

$\text{solve}(i, j)$

where:

- $i \in \{0, 1, \dots, n\}$ — current index into the **target** string (length n).
- $j \in \{0, 1, \dots, m\}$ — current index into the **typo** string (length m).

Both indices represent how far along each string we have been processed so far. The pair (i, j) uniquely identifies a subproblem.

3 Recurrence

Let $c_{\text{ins}}(i, j, \text{ch})$, $c_{\text{del}}(i, \text{ch})$, $c_{\text{sub}}(a, b)$, and $c_{\text{tr}}(a, b)$ denote the insertion, deletion, substitution, and transposition costs respectively (defined by the cost function in the project specification).

$$\text{solve}(i, j) = \min \begin{cases} \text{solve}(i+1, j+1) & \text{if } \text{target}[i] = \text{typo}[j] \\ c_{\text{ins}}(i, j, \text{typo}[j]) + \text{solve}(i, j+1) & \text{if } j < m \\ c_{\text{del}}(i, \text{target}[i]) + \text{solve}(i+1, j) & \text{if } i < n \\ c_{\text{sub}}(\text{target}[i], \text{typo}[j]) + \text{solve}(i+1, j+1) & \text{if } i < n, j < m, \text{target}[i] \neq \text{typo}[j] \\ c_{\text{tr}}(\text{target}[i], \text{target}[i+1]) + \text{solve}(i+2, j+2) & \text{if transpose-1 condition holds} \\ c_{\text{tr}}(\text{target}[i], \text{target}[i+1]) + c_{\text{tr}}(\text{target}[i], \text{target}[i+2]) + \text{solve}(i+3, j+3) & \text{if transpose-2 condition holds} \end{cases}$$

The **transpose-1 condition** requires: $i+1 < n$, $j+1 < m$, $\text{target}[i] = \text{typo}[j+1]$, and $\text{target}[i+1] = \text{typo}[j]$.

The **transpose-2 condition** requires: $i+2 < n$, $j+2 < m$, $\text{target}[i+1] = \text{typo}[j]$, $\text{target}[i+2] = \text{typo}[j+1]$, and $\text{target}[i] = \text{typo}[j+2]$.

4 Base Cases of the Recurrence

- **Both strings exhausted:** $\text{solve}(n, m) = 0$. No characters remain; no operations needed.
- **Target exhausted, typo remains** ($i = n$, $j < m$): only insertions are applicable; the recurrence naturally handles this by applying Insert repeatedly until $j = m$.
- **Typo exhausted, target remains** ($i < n$, $j = m$): only deletions are applicable; the recurrence handles this by applying Delete repeatedly until $i = n$.

No explicit boundary values other than $\text{solve}(n, m) = 0$ are needed, because the conditions on $i < n$ and $j < m$ guard each branch of the recurrence.

5 Data Structure for Recognizing Repeated Subproblems

Subproblems are uniquely identified by the pair (i, j) with $0 \leq i \leq n$ and $0 \leq j \leq m$. The implementation uses two 2-D arrays:

- **memo[n+1][m+1]** (2-D array of `int`): stores the computed minimum cost for each (i, j) once it has been solved. Initialized to $+\infty$ (`INF`).
- **seen[n+1][m+1]** (2-D array of `char`/boolean): tracks whether $\text{solve}(i, j)$ has already been computed. Initialized to `false`.
- **choice[n+1][m+1]** (2-D array of `Choice` structs): stores the operation that achieved the minimum at each state, enabling path reconstruction.

Both **memo** and **seen** are allocated as `vector<vector<...>>` of size $(n+1) \times (m+1)$. Lookups and writes are $O(1)$. Memoization ensures each state is computed at most once.

Algorithm 1 $\text{solve}(i, j)$ — Memoized Typo DP

Require: $0 \leq i \leq n, 0 \leq j \leq m$ **Ensure:** Returns minimum cost to transform $\text{target}[i:]$ into $\text{typo}[j:]$

```
1: if  $\text{seen}[i][j]$  then return  $\text{memo}[i][j]$ 
2: end if
3:  $\text{seen}[i][j] \leftarrow \text{true}$ 
4: if  $i = n$  and  $j = m$  then
5:    $\text{memo}[i][j] \leftarrow 0$ ; return 0
6: end if
7:  $\text{best} \leftarrow +\infty$ 
8: if  $i < n$  and  $j < m$  and  $\text{target}[i] = \text{typo}[j]$  then
9:    $\text{best} \leftarrow \min(\text{best}, \text{solve}(i + 1, j + 1))$  ▷ Match
10: end if
11: if  $j < m$  then
12:    $c \leftarrow c_{\text{ins}}(i, j, \text{typo}[j])$ 
13:    $\text{best} \leftarrow \min(\text{best}, c + \text{solve}(i, j + 1))$  ▷ Insert
14: end if
15: if  $i < n$  then
16:    $c \leftarrow c_{\text{del}}(i, \text{target}[i])$ 
17:    $\text{best} \leftarrow \min(\text{best}, c + \text{solve}(i + 1, j))$  ▷ Delete
18: end if
19: if  $i < n$  and  $j < m$  and  $\text{target}[i] \neq \text{typo}[j]$  then
20:    $c \leftarrow c_{\text{sub}}(\text{target}[i], \text{typo}[j])$ 
21:    $\text{best} \leftarrow \min(\text{best}, c + \text{solve}(i + 1, j + 1))$  ▷ Substitute
22: end if
23: if  $i + 1 < n$  and  $j + 1 < m$  and  $\text{target}[i] = \text{typo}[j + 1]$  and  $\text{target}[i + 1] = \text{typo}[j]$  then
24:    $c \leftarrow c_{\text{tr}}(\text{target}[i], \text{target}[i + 1])$ 
25:    $\text{best} \leftarrow \min(\text{best}, c + \text{solve}(i + 2, j + 2))$  ▷ Transpose-1
26: end if
27: if  $i + 2 < n$  and  $j + 2 < m$  and transpose-2 condition holds then
28:    $c_1 \leftarrow c_{\text{tr}}(\text{target}[i], \text{target}[i + 1])$ 
29:    $c_2 \leftarrow c_{\text{tr}}(\text{target}[i], \text{target}[i + 2])$ 
30:    $\text{best} \leftarrow \min(\text{best}, c_1 + c_2 + \text{solve}(i + 3, j + 3))$  ▷ Transpose-2
31: end if
32:  $\text{memo}[i][j] \leftarrow \text{best}$ 
33: return  $\text{best}$ 
```

6 Pseudocode for Memoized Dynamic Programming Algorithm

7 Worst-Case Time Complexity

The memoized algorithm visits each state (i, j) at most once. There are $(n+1)(m+1) = O(n \cdot m)$ states. At each state, a constant number of operations are evaluated (Match, Insert, Delete, Substitute, Transpose-1, Transpose-2), each requiring $O(1)$ work (recursive calls return immediately if already memoized; cost functions involve only constant-time keyboard-distance lookups via the pre-built `pos` hash map).

$$T(n, m) = O(n \cdot m)$$

Space complexity is also $O(n \cdot m)$ for the `memo`, `seen`, and `choice` tables, plus $O(n+m)$ recursion stack depth in the worst case.

8 Nested Loops for an Iterative DP Solution

An iterative bottom-up solution would fill the $(n+1) \times (m+1)$ table in the following order (processing *prefixes* rather than suffixes, so indices are reversed):

Listing 1: Nested-loop structure for iterative DP

```
1 // Initialize base cases
2 dp[n][m] = 0;
3 for (int i = n-1; i >= 0; --i)
4     dp[i][m] = deletionCost(i, target[i]) + dp[i+1][m];
5 for (int j = m-1; j >= 0; --j)
6     dp[n][j] = insertionCost(n, j, typo[j]) + dp[n][j+1];
7
8 // Fill remaining cells in suffix order
9 for (int i = n-1; i >= 0; --i) {
10     for (int j = m-1; j >= 0; --j) {
11         int best = INF;
12         // Match / Substitute
13         if (target[i] == typo[j])
14             best = min(best, dp[i+1][j+1]);
15         else
16             best = min(best, subCost(target[i], typo[j]) + dp[i+1][j+1]);
17         // Insert
18         best = min(best, insCost(i, j, typo[j]) + dp[i][j+1]);
19         // Delete
20         best = min(best, delCost(i, target[i]) + dp[i+1][j]);
21         // Transpose-1
22         if (i+1 < n && j+1 < m &&
23             target[i] == typo[j+1] && target[i+1] == typo[j])
24             best = min(best, trCost(target[i], target[i+1]) + dp[i+2][j
25                             +2]);
26         // Transpose-2
27         if (i+2 < n && j+2 < m && /* conditions */)
28             best = min(best, c1 + c2 + dp[i+3][j+3]);
29         dp[i][j] = best;
30     }
```

The outer loop iterates over i from $n - 1$ down to 0; the inner loop iterates over j from $m - 1$ down to 0. Every cell (i, j) depends only on cells with strictly larger i or j , so this order guarantees all dependencies are satisfied.

9 Space Complexity Improvement

The memoized (and straightforward iterative) implementation requires $O(n \cdot m)$ space for the full DP table plus the `choice` table used for reconstruction.

Can space be reduced? For *cost-only* computation (without reconstruction), the iterative variant could in principle keep only two rows of the table at a time ($O(m)$ space), since each cell (i, j) depends on $(i + 1, j)$, $(i, j + 1)$, $(i + 1, j + 1)$, $(i + 2, j + 2)$, and $(i + 3, j + 3)$ —all of which lie in the next one to three rows. Three rows would suffice if reconstruction is not needed.

However, reconstruction requires backtracking through the `choice` table, which cannot be discarded. Therefore, with reconstruction the full $O(n \cdot m)$ `choice` table must be retained, and the overall space complexity remains $O(n \cdot m)$. The memoized implementation carries an additional $O(n + m)$ stack overhead that the iterative version avoids.

10 Advantages and Disadvantages of the Iterative Algorithm

Advantages

- **No recursion overhead.** The iterative approach eliminates function-call stack frames and avoids the risk of stack overflow for large inputs (strings up to 1 000 characters).
- **Predictable memory layout.** Sequential row-by-row access gives better CPU cache locality than the order-dependent call graph of memoized recursion.
- **Simpler termination.** The loop structure makes it obvious that every state is visited exactly once; with memoization one must reason about the DAG of recursive calls.
- **Easier to optimise further.** Early-termination heuristics (e.g., pruning when a running cost already exceeds a known bound) slot naturally into the loop body.

Disadvantages

- **Must compute all states.** The iterative approach fills every cell even if many are unreachable. The memoized version naturally skips states that are never called, which can be beneficial when the optimal path is narrow.
- **Harder to extend.** Adding new operation types (e.g., a longer look-ahead transpose) requires carefully adjusting the loop bounds and dependency order, whereas the recursive version only needs a new branch in `solve`.
- **Reconstruction logic is separate.** The iterative approach must maintain a parallel `choice` table; the memoized version fills both tables together naturally.

Which is better overall?

For this project the **memoized** implementation is preferable. The code is more readable and easier to verify correct because the recursive structure mirrors the mathematical recurrence directly. For inputs up to 1 000 characters the recursion depth is at most $n + m \leq 2\,000$, well within typical

stack limits. Both approaches have the same $O(n \cdot m)$ time and space complexity, so there is no asymptotic reason to prefer the iterative version. The iterative approach would become necessary only if inputs were large enough to risk stack overflow or if performance profiling revealed cache-miss bottlenecks in practice.